Model Deployment Task

Serving a Trained Model via FastAPI and Docker

---

1. Objective

Wrap a trained machine learning model in a lightweight REST API and package it as a Docker container so it can be deployed consistently across environments. The API exposes endpoints for single and batch inference and includes interactive documentation, automated tests, and ready-to-use example requests.

2. Code & Resources

Item	Location
GitHub repository	<your-repo-url> (push the project folder below)
API source code	`app/main.py, app/schemas.py`
Model artifact	`app/model.joblib`
Training script	`train_model.py`
Dockerfile	`Dockerfile, .dockerignore`
Tests	`tests/test_api.py`
Example requests	`examples/*.json, examples/curl_examples.sh`
Setup guide	`README.md`

3. Approach & Architecture

3.1 Model

A RandomForestClassifier is trained on the scikit-learn Iris dataset (150 samples, 4 numeric features, 3 target classes) using train_model.py. The fitted model, along with feature names and target class labels, is serialized with joblib to app/model.joblib. This artifact is bundled directly into the Docker image so the container is self-sufficient at runtime.

3.2 API Layer (FastAPI)

The API is implemented with FastAPI (app/main.py) and uses Pydantic models (app/schemas.py) for request/response validation. Endpoints:

- **GET /** - basic API info and a list of available endpoints
- **GET /health** - health check, reports whether the model loaded successfully

- **POST /predict** - returns the predicted class and class probabilities for a single set of features
- **POST /predict/batch** - same as above, but accepts a list of feature sets and returns a list of predictions

The model is loaded once at application startup via a FastAPI lifespan handler, avoiding repeated disk reads on every request. Invalid payloads automatically return HTTP 422 thanks to Pydantic validation, and a missing model returns HTTP 503.

3.3 Containerization (Docker)

The Dockerfile uses a slim Python 3.11 base image, installs dependencies from requirements.txt, copies the application code and model artifact, and runs the app with uvicorn on port 8000. A HEALTHCHECK polls the /health endpoint.

```
FROM python:3.11-slim
WORKDIR /code
COPY requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir -r /code/requirements.txt
COPY app /code/app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. How to Run

4.1 Local (Python virtual environment)

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python train_model.py            # (optional) retrain the model
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Interactive Swagger documentation is then available at http://localhost:8000/docs

4.2 Docker

```
docker build -t iris-classifier-api .
docker run -d -p 8000:8000 --name iris-api iris-classifier-api
curl http://localhost:8000/health
```

5. Example Request & Response

Single prediction - POST /predict

```
# Request
{ "sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width": 0.2
}


# Response
{ "predicted_class": "setosa", "predicted_class_index": 0,
  "probabilities": {"setosa": 1.0, "versicolor": 0.0, "virginica": 0.0} }
```

Batch prediction - POST /predict/batch

```
{ "instances": [
  {"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width":
0.2},
  {"sepal_length": 6.7, "sepal_width": 3.0, "petal_length": 5.2, "petal_width":
```

```
2.3}
] }

# Response (excerpt)
{ "predictions": [
  {"predicted_class": "setosa", "predicted_class_index": 0, ...},
  {"predicted_class": "virginica", "predicted_class_index": 2, ...}
] }
```

6. Demo Screenshot

Live curl requests against the running API, showing the health check, single prediction, and batch prediction responses:

7. Testing

Automated tests (tests/test_api.py) use FastAPI's TestClient to verify the root, health, single, and batch prediction endpoints, including validation error handling. Run with:

```
pip install pytest httpx
pytest tests/ -v
```

Result: 7 passed (root, health, setosa prediction, virginica prediction, batch prediction, empty batch rejection, invalid payload rejection).

8. Notes & Extensibility

- To use a different model, retrain or replace app/model.joblib and update app/schemas.py to match the new feature set and output classes.
- The container exposes port 8000 and includes a Docker HEALTHCHECK against /health for orchestration platforms (e.g. Docker Compose, Kubernetes).
- Full setup, environment, and run instructions are documented in README.md in the project root.